

CSE 144 Final Project Report

Transfer Learning Challenge

Jacob Johnston (CruzID: 2247649)

Spring 2026

1 Introduction

The task is 100-class image classification with roughly 10 labeled training images per class (1,079 images total) and 1,036 unlabeled test images scored on Kaggle. With about ten examples per class, training a network from scratch is hopeless: a modern vision backbone has 10^7 – 10^8 parameters, and the dataset cannot constrain them. Transfer learning is the natural fit: a backbone pretrained on data already encodes general visual features, so the task reduces to learning a 100-way decision rule on top of those features and, at most, lightly adapting the last few layers.

Our approach uses CLIP image encoders from `open_clip` [3, 1] with a two-phase *linear-probe-then-fine-tune* (LP-FT) procedure [6]: first train a linear head on cached frozen embeddings, then unfreeze only the last two transformer blocks at a small learning rate. All design decisions, such as backbone, unfreezing depth, backbone learning rate, and epoch budgets, were made with stratified 4-fold cross-validation rather than the public leaderboard, using a one-standard-error rule as a guide for configuration selection and explicit CV thresholds as gates before scaling up the backbone. The final submission is a 4-fold checkpoint ensemble of ViT-L/14 (LAION-2B pretraining), which reached **0.91818** on the public leaderboard (top 10 at the time of writing) from a cross-validation estimate of 0.8925 ± 0.0069 .

2 Dataset

Classes and sizes. 100 classes stored as `train/0/` ... `train/99/`, with 1,079 training images in total. Class sizes are approximately 10 but not uniform; the smallest class (42) has only 4 images, which caps stratified cross-validation at $K=4$ folds. The test set contains 1,036 images. Validation sets are the CV folds (≈ 270 images each); every training image appears in exactly one fold’s validation split.

Label mapping. Folder names are the integer labels, so we build the sample list by iterating `range(100)` and mapping folder `str(class_id)` $\rightarrow$ label `class_id` directly. This matters: generic loaders that sort class folders as strings (0, 1, 10, 100, ...) scramble the mapping. Our very first submission had a related label-space bug which emitted raw ImageNet class indices (values up to 998; 26% of predictions outside 0–99) and scored 0.118, which set the lesson early.

Preprocessing. Each backbone uses its paired `open_clip` transform: resize and center-crop to 224×224 , then CLIP normalization. Inputs at inference are preprocessed identically.

Augmentation. We evaluated horizontal-flip augmentation only (CLIP-safe; no color jitter or random crops that fight the pretraining distribution). It *reduced* accuracy (Section 4), so the final model trains without augmentation.

3 Implementation

3.1 Model

Backbone. `open_clip` ViT-L/14 [2] pretrained on LAION-2B [4, 5] (`laion2b_s32b_b82k`), producing 768-d image embeddings. The choice was empirical: at identical architecture, LAION-2B checkpoints beat OpenAI checkpoints by +5.7 points in head-only CV (ViT-B/16: 0.850 vs. 0.793), and L/14 beat B/16 by a further +3.2 points before any fine-tuning (Table 1).

Head. Dropout(0.3) followed by a single `Linear(768 → 100)` layer. No intermediate MLP — with ~ 8 training images per class per fold, extra head capacity only adds variance.

Fine-tuning strategy (LP-FT). Phase A trains the head alone on embeddings precomputed once with the frozen backbone (so each epoch costs milliseconds). Phase B then unfreezes the *last 2 of 24* transformer blocks plus the final layer norm and projection, at a backbone learning rate of 10^{-5} while the head continues at 10^{-3} . Initializing fine-tuning from a converged linear probe avoids the feature distortion that a randomly initialized head causes early in training [6]. The unfreeze depth and learning rate were selected by CV (Section 4), not assumed.

3.2 Training

Loss and optimizer. Cross-entropy with label smoothing 0.1; AdamW with weight decay 10^{-4} .

Schedules. Head LR 10^{-3} (both phases); backbone LR 10^{-5} with cosine annealing ($T_{\max}=30$) in Phase B. Early stopping on fold-validation accuracy with patience 10 (Phase A) and 5 (Phase B), restoring the best-validation weights. Batch size 64 for ViT-B/16, 32 for ViT-L/14 (to stay within GPU memory during fine-tuning). Maximum epochs: 60 (Phase A), 30 (Phase B); in practice Phase B converges in 1–8 epochs.

Environment. Windows, NVIDIA GeForce RTX 3060 (CUDA), PyTorch 2.11.0+cu128 [8] with `open_clip_torch` 3.3.0, scikit-learn 1.9.0 [9] for stratified splits.

4 Experiments

Baselines. An ImageNet-pretrained CNN baseline scored 0.118 due to the label-space bug described above. Early single-split CLIP ViT-B/32 experiments with a frozen backbone and progressive unfreezing landed at 0.709–0.736 on the public leaderboard, and motivated three changes: a proper CV protocol, stronger pretraining data, and disciplined unfreezing.

Validation protocol. Stratified 4-fold CV (seed 42; $K=4$ forced by the smallest class). Within a configuration, each fold runs Phase A and Phase B with early stopping; the configuration’s score is the mean of fold-best validation accuracies. Two decision rules kept us honest:

- *One-SE rule for configuration selection.* Among unfreezing configurations, select the simplest whose CV mean is within one standard error ($SE = \text{std}/\sqrt{K}$) of the best mean [7].
- *Gates for scaling up.* A larger backbone earns a (costly) fine-tuning CV run only if its *head-only* CV mean exceeds the incumbent’s by more than $2 \times SE$, and earns a test submission only if its fine-tuned CV mean clears the incumbent’s by the same margin. L/14 passed both gates ($0.8823 > 0.8583$; $0.8925 > 0.8765$).

Ablations. (i) *Backbone and pretraining data:* Table 1. (ii) *Unfreeze depth and backbone LR:* four configurations on ViT-B/16, Table 2. (iii) *Augmentation:* horizontal flip cost 6.4 public points on the early B/32 setup ($0.736 \rightarrow 0.673$). (iv) *Test-time augmentation:* horizontal-flip TTA was

neutral (0.709). (v) *Training on all data without validation*: cost 1.8 points ($0.736 \rightarrow 0.718$) for lack of an early-stopping signal — the fix in the final pipeline is to retrain on all data using the CV-median best epoch as a fixed budget (18 epochs Phase A, 2 epochs Phase B for B/16).

5 Results

Table 1: Head-only (linear probe) 4-fold CV by backbone. LAION-2B pretraining and scale both help before any fine-tuning.

Backbone	Pretraining	Embed dim	CV accuracy (mean $\pm$ std)
ViT-B/32	OpenAI	512	0.7553 ± 0.0117
ViT-B/16	OpenAI	512	0.7933 ± 0.0017
ViT-B/16	LAION-2B	512	0.8499 ± 0.0085
ViT-L/14	LAION-2B	768	0.8823 ± 0.0109

Table 2: Unfreezing configurations on ViT-B/16 (LAION-2B), 4-fold CV. The one-SE rule (threshold $0.8647 - 0.0071 = 0.8575$) selects `unfreeze_2_1e5` as the simplest candidate within one SE of the best mean.

Config	Blocks unfrozen	Backbone LR	CV mean	CV std	Median Phase-B epoch
<code>head_only</code>	0	—	0.8499	0.0085	—
<code>unfreeze_2_1e6</code>	2	10^{-6}	0.8545	0.0144	3
<code>unfreeze_2_1e5</code>	2	10^{-5}	0.8619	0.0145	2
<code>unfreeze_4_1e5</code>	4	10^{-5}	0.8647	0.0143	2

Applying the selected configuration (`unfreeze_2`, backbone LR 10^{-5}) to ViT-L/14 gives fold validation accuracies 0.8889/0.9037/0.8852/0.8922, i.e. **0.8925 ± 0.0069** (a +3.1-point CV improvement over the same recipe on B/16, consistent with the head-only gap).

Final predictor. Rather than retraining on all data and discarding the fold models, the final submission ensembles the four L/14 fold checkpoints by summing their logits over the test set and taking the argmax. Each fold model agrees with the ensemble on 93–95% of test images; the ensemble recovers the accuracy lost to each fold’s reduced ($\sim 75\%$) training set while using every training image.

The public score (0.918) sits within one fold-std of the CV estimate (0.8925), and ranked top-10 on the public leaderboard at the time of writing.

6 Discussion

What worked. (1) *Pretraining data quality*: at fixed architecture, swapping OpenAI for LAION-2B weights was worth +5.7 CV points. This is the single largest gain in the project. (2) *Scale, but gated*: L/14 added +3 points over B/16, and the CV gates meant we paid its compute cost only after the cheap head-only check justified it. (3) *LP-FT with conservative unfreezing*: Phase B improves CV by ~ 1 point and converges in a median of 2 epochs — fine-tuning here is a small correction on top of strong features, and unfreezing deeper (4 blocks) buys nothing the one-SE rule can distinguish from noise. (4) *Fold ensembling*: +0.9 public points over the single full-data model, essentially

Table 3: Public leaderboard progression ($\approx 11\%$ of test, 110 images; one image ≈ 0.9 points).

#	Submission	Public score
1	ImageNet-pretrained baseline (label-space bug)	0.118
2	CLIP ViT-B/32, frozen backbone + linear head	0.709
3	+ progressive unfreezing	0.727
4	+ more epochs	0.736
5	trained on all data, no validation split	0.718
6	+ horizontal-flip augmentation	0.673
7	+ horizontal-flip TTA	0.709
8	ViT-B/16 LAION-2B, unfreeze.2.1e5, full-data retrain (CV epoch budget)	0.864
9	ViT-B/16 LAION-2B, 4-fold ensemble	0.873
10	ViT-L/14 LAION-2B, 4-fold ensemble (final)	0.918

free given the CV checkpoints already exist, and it sidesteps choosing an epoch budget without validation data.

What failed. Horizontal-flip augmentation hurt badly on this dataset; with ~ 8 images per class per fold and a test set drawn from the same distribution as training, we attribute the drop to the train/inference input mismatch flipping introduces (test images are never flipped) outweighing its regularization benefit. Training on all data without a validation split underperformed for lack of a stopping signal. TTA was neutral.

Limitations. One seed per fold (fold-level variance is measured, seed-level is not). The public leaderboard is 110 images, so public differences under ~ 2 points are noise; we therefore avoided selecting on it. Architecture-level decisions reused the same four folds across stages, so the final CV estimate carries mild selection optimism. The unfreezing configuration was selected on B/16 and transferred to L/14 rather than re-researched at scale; L/14’s CV validates the transferred recipe, not its optimality. **Next steps:** out-of-fold-validated blending of the B/16 and L/14 ensembles (softmax-probability averaging across all eight checkpoints), multi-seed folds, and a per-class error analysis of the $\sim 8\%$ the ensemble still misses.

7 Reproducibility

Seeds and determinism. Global seed 42 for Python/NumPy/PyTorch; fold i reseeds with $42 + i$ before training. The `StratifiedKFold(n_splits=4, shuffle=True, random_state=42)` split is fully deterministic given the sorted directory traversal, so fold membership is identical across runs. cuDNN determinism flags are not forced, so GPU runs can vary by a fraction of a point; reported CV numbers are means over 4 folds.

Environment. Python 3.12.10; torch==2.11.0+cu128, open_clip_torch==3.3.0, scikit-learn==1.9.0, pandas, Pillow, tqdm. Install with `pip install -r requirements.txt`.

To reproduce training. Place data under `./data/train` and `./data/test`. Run `final_clip_kfold.ipynb` top to bottom: the B/16 cells run the configuration CV and selection; the Stage-1 cell runs the L/14 head-only gate; Stage 2 (`RUN_L14_UNFREEZE=True`) trains the four L/14 fold models and writes `checkpoints_kfold/l14_unfreeze_2.1e5_fold{0..3}_best.pt`.

To reproduce the submission from provided weights. Download the four L/14 fold checkpoints from the Google Drive link in the README into `checkpoints_kfold/`, then run the Stage-3 cell (`RUN_L14_SUBMISSION=True`), which writes `submission_l14_ensemble.csv` (1,036 rows,

columns ID,Label).

8 Team Contributions

- Jacob Johnston: all components (data pipeline, CV framework, backbone/configuration experiments, ensembling, report).

9 References

References

- [1] A. Radford et al. “Learning Transferable Visual Models From Natural Language Supervision.” *ICML*, 2021.
- [2] A. Dosovitskiy et al. “An Image is Worth 16×16 Words: Transformers for Image Recognition at Scale.” *ICLR*, 2021.
- [3] G. Ilharco et al. “OpenCLIP.” Zenodo, 2021. https://github.com/mlfoundations/open_clip
- [4] C. Schuhmann et al. “LAION-5B: An open large-scale dataset for training next generation image-text models.” *NeurIPS Datasets and Benchmarks*, 2022.
- [5] M. Cherti et al. “Reproducible scaling laws for contrastive language-image learning.” *CVPR*, 2023.
- [6] A. Kumar et al. “Fine-Tuning can Distort Pretrained Features and Underperform Out-of-Distribution.” *ICLR*, 2022.
- [7] T. Hastie, R. Tibshirani, J. Friedman. *The Elements of Statistical Learning*, 2nd ed., Springer, 2009 (one-standard-error rule, §7.10).
- [8] A. Paszke et al. “PyTorch: An Imperative Style, High-Performance Deep Learning Library.” *NeurIPS*, 2019.
- [9] F. Pedregosa et al. “Scikit-learn: Machine Learning in Python.” *JMLR*, 12:2825–2830, 2011.